ServiX

Gestión de clientes, servicios y atención con IA

PLAN DE PRUEBAS

Plan de Pruebas de ServiX

Estrategia, alcance, entorno, criterios y procedimientos de prueba de la aplicación Android ServiX

Proyecto	ServiX — Gestión de clientes, servicios y atención con IA
Código del documento	DOC-03
Versión	1.0
Fecha	01/11/2026
Estándar de referencia	ISO/IEC/IEEE 29119-3:2021 (plan de pruebas)
Institución	Universidad de San Buenaventura
Asignatura	Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Nicolas Quiroz Gomez	Product Owner
Juan Sebastian Garzon Suarez	Scrum Master
Juan Esteban Olaya Muñoz	Developer

Control del documento

Información general

Título	Plan de Pruebas de ServiX
Código	DOC-03
Versión	1.0
Fecha de emisión	01/11/2026
Estado	Versión 1.0 emitida; aprobaciones pendientes
Proyecto	ServiX
Autor	Equipo ServiX
Clasificación	Uso académico; sin información confidencial

Historial de revisiones

Versión	Fecha	Descripción del cambio	Autor
0.9	01/11/2026	Borrador inicial en formato plano (archivado como respaldo v1).	Equipo ServiX
1.0	01/11/2026	Rediseño completo: portada, control de versiones, numeración, figuras y tablas rotuladas, mayor profundidad.	Equipo ServiX

Aprobaciones

Rol	Nombre	Fecha	Firma / estado
Elaboró	Juan Esteban Olaya Muñoz (Developer)	Pendiente	Pendiente
Revisó	Juan Sebastian Garzon Suarez (Scrum Master)	Pendiente	Pendiente
Aprobó	Nicolas Quiroz Gomez (Product Owner)	Pendiente	Pendiente

Las aprobaciones se encuentran **pendientes**: no se registran firmas ni fechas de aprobación hasta que los responsables las emitan.

Tabla de contenido

1 Introducción **5**
1.1 Propósito 5
1.2 Alcance del documento 5
1.3 Referencias 5
2 Política de pruebas **6**
3 Estrategia de pruebas **7**
3.1 Niveles y tipos 7
3.2 Enfoque 7
3.3 Diseño de casos 7
4 Alcance de las pruebas **8**
5 Entorno de pruebas **9**
6 Herramientas **10**
7 Criterios **11**
7.1 Criterios de entrada 11
7.2 Criterios de salida 11
7.3 Criterios de suspensión y reanudación 11
8 Datos de prueba **12**
9 Roles y responsabilidades **13**
10 Calendario **14**
11 Riesgos de las pruebas **15**
12 Matriz de cobertura requisito-caso **16**
13 Procedimiento de ejecución **18**
13.1 Pruebas locales 18
13.2 Pruebas en Azure Pipelines 18
13.3 Pruebas pendientes 18
Anexo A. Catálogo de casos de prueba **19**

Lista de tablas

Tabla 1. Documentos de referencia 5
Tabla 2. Niveles y tipos de prueba 7
Tabla 3. Elementos dentro y fuera del alcance 8
Tabla 4. Entornos de prueba 9
Tabla 5. Herramientas de prueba y versiones 10
Tabla 6. Criterios de salida por nivel 11
Tabla 7. Suspensión y reanudación 11
Tabla 8. Datos de prueba 12
Tabla 9. Roles y responsabilidades de las pruebas 13
Tabla 10. Calendario de pruebas por Sprint (fechas provisionales) 14
Tabla 11. Riesgos del proceso de pruebas 15
Tabla 12. Cobertura de requisitos funcionales por casos de prueba 16
Tabla 13. Cobertura de requisitos no funcionales 17
Tabla 14. Procedimiento de las pruebas pendientes 18
Tabla 15. Catálogo de casos de prueba (CP-001 a CP-066) 19

1 Introducción

1.1 Propósito

Este plan define cómo se verifica ServiX: qué se prueba, con qué técnicas y herramientas, en qué entorno, con qué criterios se acepta el resultado y quién es responsable de cada actividad. Es el documento de referencia del Informe de Pruebas (DOC-04), que registra los resultados reales de la ejecución.

1.2 Alcance del documento

Cubre las pruebas de la aplicación Android (app), de sus reglas de seguridad de Firestore y del pipeline de integración continua. El plan distingue explícitamente entre lo que ya se ejecutó y lo que sigue pendiente, de modo que ninguna actividad se declara como realizada sin evidencia.

1.3 Referencias

Tabla 1. Documentos de referencia

Código	Documento	Uso
DOC-01	SRS de ServiX	Origen de los requisitos a verificar.
DOC-02	Arquitectura y Diseño	Capas y puntos de prueba.
DOC-04	Informe de Pruebas	Resultados de la ejecución.
—	`Casos_Prueba.xlsx` y `Matriz_Maestra.xlsx`	Catálogo completo de casos y trazabilidad.
—	`azure-pipelines.yml`	Definición del pipeline de CI.

2 Política de pruebas

El equipo adopta las siguientes reglas de calidad, aplicadas a cada Pull Request hacia `main`:

- Todo cambio en la lógica de dominio debe acompañarse de pruebas unitarias en la JVM o justificar su ausencia.
- Las pruebas deben ser deterministas y ejecutables sin red: se usan repositorios simulados en lugar de Firebase.
- El pipeline de CI debe pasar (pruebas y compilación) antes de integrar en `main`.
- No se eliminan ni se desactivan pruebas para conseguir una ejecución en verde.
- Lo que no se ejecutó se registra como **No ejecutado** con su motivo; no se reporta como aprobado.
- Las pruebas no deben contener secretos, correos reales ni datos personales; se emplean datos ficticios.

3 Estrategia de pruebas

3.1 Niveles y tipos

Tabla 2. Niveles y tipos de prueba

Nivel / tipo	Objetivo	Técnica y herramienta	Estado actual
Unitaria (dominio)	Casos de uso y modelos con repositorios simulados.	JUnit 4, corrutinas de prueba.	Ejecutada (18 pruebas).
Integración con doble	Casos de uso con un webhook de Make simulado.	JUnit con clase `Fake`.	Ejecutada (3 pruebas).
Interfaz sobre JVM	Lanzamiento, navegación y captura de pantalla.	Robolectric, Compose UI Test y Roborazzi.	Ejecutada (8 pruebas).
Compilación	Verificar que el APK de depuración se construye.	`gradlew assembleDebug`.	Ejecutada.
Integración continua	Compilar y probar en un entorno limpio.	Azure Pipelines `ServiX-CI`.	Ejecutada (run 20261101.3).
Análisis estático	Detectar problemas de calidad.	Android Lint (`lintDebug`).	**No ejecutada**.
Instrumentada	Pruebas en dispositivo o emulador.	`connectedDebugAndroidTest`.	**No ejecutada**.
Funcional en dispositivo	Flujos por rol, datos reales y usabilidad.	Manual con APK de depuración.	**No ejecutada**.
Integración externa	Firebase Auth, Firestore, Gemini y Make reales.	Manual y emulador de Firebase.	**No ejecutada**.
Seguridad de reglas	Denegación de accesos no autorizados.	Emulador de reglas de Firestore o consola.	**No ejecutada**.

3.2 Enfoque

La pirámide de pruebas del proyecto es ancha en la base y estrecha en la cima: la mayoría de las pruebas automatizadas se concentran en el dominio, que es donde está la lógica y donde se pueden ejecutar en milisegundos. Las pruebas de interfaz con Robolectric validan el arranque y la navegación sin necesidad de dispositivo, lo que es crítico dado que el equipo de desarrollo no admite virtualización. Las pruebas que necesitan servicios reales se reservan para una fase posterior con dispositivo.

3.3 Diseño de casos

- Casos normales: alta de cliente, cambio de estado de servicio, inicio de sesión válido.
- Entradas inválidas: contraseña de menos de 6 caracteres.
- Casos límite: cálculo de iniciales con una, dos y tres palabras; eliminación de un cliente con servicios asociados.
- Integración: evento enviado a Make al crear y al completar un servicio.
- Navegación: cada destino de la barra inferior muestra su pantalla.

4 Alcance de las pruebas

Tabla 3. Elementos dentro y fuera del alcance

Dentro del alcance	Fuera del alcance
Casos de uso del dominio y modelos.	Rendimiento y pruebas de carga.
Repositorios simulados y flujos de datos.	Accesibilidad con lectores de pantalla.
Navegación de la interfaz con Robolectric.	Compatibilidad con múltiples fabricantes y versiones de Android.
Pipeline de CI y compilación del APK.	Pruebas de penetración.
Reglas de Firestore (cuando se ejecute el emulador).	Pruebas con usuarios finales.
Integraciones reales con Firebase, Gemini y Make (fase pendiente).	Firma y publicación de versiones *release*.

5 Entorno de pruebas

Tabla 4. Entornos de prueba

Entorno	Configuración	Uso
Equipo local de desarrollo	Windows 10, 4 GB de RAM, sin virtualización, Gradle con wrapper del repositorio, JDK 21.	Pruebas unitarias y Robolectric; compilación (más de 20 minutos por la memoria disponible).
Azure Pipelines	Agente alojado `ubuntu-latest`, JDK 21, Android SDK 36.1, caché de Gradle.	CI en cada cambio a `main` y en Pull Requests.
Dispositivo físico (pendiente)	Teléfono Android 8.0 o superior con el APK de depuración.	Pruebas funcionales y de integración.
Proyecto Firebase de pruebas (pendiente)	Authentication, Firestore y AI Logic habilitados; cuentas ficticias.	Pruebas con servicios reales.

Advertencia. JDK 21 obligatorio para las pruebas
Robolectric con SDK 36 exige Java 21. Con JDK 17 las pruebas de interfaz fallan; este fue el defecto real del run 2 del pipeline y está corregido en `azure-pipelines.yml` (véase DOC-04).

6 Herramientas

Tabla 5. Herramientas de prueba y versiones

Herramienta	Versión	Propósito
JUnit	4.13.2	Marco de pruebas unitarias.
Robolectric	4.16.1	Android en la JVM.
Roborazzi	1.59.0	Capturas de pantalla de Compose.
Compose UI Test (BOM 2024.09.00)	—	Verificación de nodos de interfaz.
Corrutinas de prueba	1.10.2	Pruebas de código asíncrono.
Gradle / AGP / Kotlin	AGP 9.1.1, Kotlin 2.2.10	Compilación y ejecución.
Azure Pipelines	—	CI y publicación de resultados JUnit.

7 Criterios

7.1 Criterios de entrada

- El código compila (`assembleDebug`).
- El repositorio no contiene secretos.
- Los requisitos de la funcionalidad están en el SRS.
- Para pruebas con servicios reales: proyecto Firebase y cuentas de prueba disponibles.

7.2 Criterios de salida

Tabla 6. Criterios de salida por nivel

Nivel	Criterio de salida
Unitaria e integración con doble	100 % de las pruebas aprobadas; sin pruebas desactivadas.
Interfaz sobre JVM	100 % aprobadas con JDK 21.
CI	Pipeline en verde y artefacto `servix-apk-debug` publicado.
Funcional en dispositivo	Casos CP-030 a CP-048 ejecutados; defectos críticos y altos corregidos.
Reglas de seguridad	Accesos no autorizados denegados en el emulador de reglas.

7.3 Criterios de suspensión y reanudación

Tabla 7. Suspensión y reanudación

Condición de suspensión	Condición de reanudación
El proyecto no compila.	Compilación exitosa del APK de depuración.
Fallo del pipeline por configuración del entorno (JDK, SDK, dependencias).	Entorno corregido y run exitoso.
Cuota o disponibilidad de Firebase o Gemini agotadas.	Servicio restablecido o cuenta de pruebas alternativa.
Se detecta una vulnerabilidad crítica de seguridad.	Vulnerabilidad corregida y verificada.

8 Datos de prueba

Las pruebas automatizadas emplean los datos ficticios de `MockDataStore` (clientes y servicios de ejemplo) y datos construidos en cada prueba. Las pruebas en dispositivo deben usar cuentas creadas expresamente para ese fin.

Tabla 8. Datos de prueba

Dato	Origen	Restricción
Clientes y servicios iniciales	`MockDataStore`	Datos ficticios.
Usuario inicial de prueba	`MockAuthRepository`	Solo en memoria.
Cuentas de Firebase para pruebas	Creadas por el equipo en el proyecto de pruebas	No se registran contraseñas en el repositorio ni en documentos.
Mensajes de chat	Textos con palabras clave de queja, cancelación, problema o consulta general	Sin datos personales.
URL de Make de prueba	`local.properties` o `MAKE_WEBHOOK_URL`	Nunca se versiona.

9 Roles y responsabilidades

Tabla 9. Roles y responsabilidades de las pruebas

Rol	Persona	Responsabilidad
Developer (autor de pruebas)	Juan Esteban Olaya Muñoz	Diseñar, implementar y ejecutar las pruebas; documentar resultados.
Scrum Master	Juan Sebastian Garzon Suarez	Velar por el pipeline y por que las pruebas se ejecuten en cada Sprint.
Product Owner	Nicolas Quiroz Gomez	Validar que los criterios de aceptación reflejan el valor esperado.
Docente	Ing. Jairo Armando Salcedo Aranda	Evaluar el proceso y los entregables.

10 Calendario

El calendario se basa en los tres Sprints configurados en Azure DevOps. Las fechas son **provisionales** y deben confirmarse con el calendario académico.

Tabla 10. Calendario de pruebas por Sprint (fechas provisionales)

Sprint	Fechas	Actividades de prueba	Entregables
Sprint 1	02/11/2026 – 13/11/2026	Planificación de pruebas, revisión de requisitos y ejecución de pruebas existentes.	Plan de Pruebas; línea base de resultados.
Sprint 2	16/11/2026 – 27/11/2026	Pruebas funcionales en dispositivo por rol; pruebas con Firebase y Gemini reales; reglas de Firestore.	Casos CP-030 a CP-043 ejecutados.
Sprint 3	30/11/2026 – 11/12/2026	Lint, pruebas instrumentadas, regresión, informe final.	Informe de Pruebas definitivo.

11 Riesgos de las pruebas

Tabla 11. Riesgos del proceso de pruebas

ID	Riesgo	Mitigación
RP-01	Equipo lento (4 GB de RAM; compilación de más de 20 minutos).	Apoyarse en Azure Pipelines; ejecutar subconjuntos de pruebas.
RP-02	Sin dispositivo ni emulador: las pruebas funcionales no se ejecutan.	Conseguir un teléfono físico para el Sprint 2.
RP-03	Cambios de versión de la imagen `ubuntu-latest`.	Fijar una versión estable de imagen.
RP-04	Cuotas y costos del uso de Gemini.	Usar pocas consultas, configurar cuotas.
RP-05	Invitaciones pendientes de aceptar en Azure DevOps.	Confirmar el acceso de los integrantes.
RP-06	Pruebas de interfaz sensibles al entorno (JDK y SDK).	Mantener JDK 21 en el pipeline y documentar el requisito.

12 Matriz de cobertura requisito-caso

La tabla muestra, para cada requisito funcional, los casos de prueba que lo cubren y su estado actual. Los casos CP-001 a CP-029 son las pruebas automatizadas; CP-030 a CP-048 están pendientes; CP-049 y CP-050 verifican compilación y CI; CP-051 a CP-066 son renders de pantallas con datos simulados.

Tabla 12. Cobertura de requisitos funcionales por casos de prueba

RF	Requisito	Casos	Ejecución
RF-001	Registro de cuenta	CP-016, CP-033	1 de 2 ejecutados
RF-002	Inicio de sesión con validación de rol	CP-014, CP-015, CP-030, CP-031, CP-032	2 de 5 ejecutados
RF-003	Cierre de sesión	CP-017	1 de 1 ejecutados
RF-004	Recuperación de contraseña	CP-034	0 de 1 ejecutados
RF-005	Navegación según rol	CP-003, CP-004, CP-025, CP-026, CP-027, CP-028, CP-029, CP-030, CP-031	7 de 9 ejecutados
RF-006	Gestión de clientes: alta y edición	CP-005, CP-006, CP-007, CP-011, CP-012, CP-035	5 de 6 ejecutados
RF-007	Eliminación y restauración de clientes	CP-009, CP-035	1 de 2 ejecutados
RF-008	Favoritos	CP-035, CP-036	0 de 2 ejecutados
RF-009	Segmentación automática de clientes	CP-035	0 de 1 ejecutados
RF-010	Gestión de servicios	CP-005, CP-008, CP-010, CP-012, CP-036	4 de 5 ejecutados
RF-011	Seguimiento del estado del servicio	CP-008, CP-012, CP-023, CP-036	3 de 4 ejecutados
RF-012	Historial de actividades y resumen financiero	CP-028, CP-037	1 de 2 ejecutados
RF-013	Notificaciones internas	CP-018, CP-037	1 de 2 ejecutados
RF-014	Búsqueda global	CP-037	0 de 1 ejecutados
RF-015	Panel de inicio y metas semanales	CP-025	1 de 1 ejecutados
RF-016	Perfil del negocio y configuración	CP-029, CP-046	1 de 2 ejecutados
RF-017	Tutorial de bienvenida	CP-025	1 de 1 ejecutados
RF-018	Asistente de chat con IA	CP-038, CP-048	0 de 2 ejecutados
RF-019	Clasificación de intención y escalamiento	CP-018, CP-039	1 de 2 ejecutados
RF-020	Gestión de solicitudes de atención (administrador)	CP-018, CP-019, CP-020, CP-021, CP-041	4 de 5 ejecutados
RF-021	Portal del cliente	CP-040	0 de 1 ejecutados
RF-022	Notificación de eventos a Make	CP-022, CP-023, CP-024, CP-043	3 de 4 ejecutados
RF-023	Restablecimiento de datos	CP-029	1 de 1 ejecutados

Tabla 13. Cobertura de requisitos no funcionales

RNF	Categoría	Casos o método
RNF-001	Seguridad	CP-040, CP-042
RNF-002	Seguridad	Inspección de código
RNF-003	Seguridad	Inspección de código
RNF-004	Seguridad	Inspección de código
RNF-005	Compatibilidad	CP-047
RNF-006	Mantenibilidad	CP-044
RNF-007	Usabilidad	CP-046, CP-051, CP-052, CP-053, CP-054, CP-055, CP-056, CP-057, CP-058, CP-059, CP-060, CP-061, CP-062, CP-063, CP-064, CP-065, CP-066
RNF-008	Fiabilidad	CP-048
RNF-009	Testabilidad	CP-001, CP-002, CP-045, CP-050
RNF-010	Portabilidad y CI	CP-049, CP-050
RNF-011	Persistencia	Inspección de código
RNF-012	Mantenibilidad	Inspección de código

13 Procedimiento de ejecución

13.1 Pruebas locales

Comandos en Windows (símbolo del sistema o PowerShell)

```
gradlew.bat testDebugUnitTest
gradlew.bat assembleDebug
```

Comandos en Linux y macOS

```
./gradlew testDebugUnitTest
./gradlew assembleDebug
```

Los resultados quedan en `app/build/test-results/testDebugUnitTest/` (XML de JUnit) y en el informe HTML de Gradle. En el equipo de referencia conviene cerrar otras aplicaciones antes de ejecutar: la memoria es el principal limitante.

13.2 Pruebas en Azure Pipelines

1. Crear una rama de funcionalidad (`feature/...`), confirmar los cambios y subirla.
2. Abrir un Pull Request hacia `main`; el pipeline `ServiX-CI` se ejecuta automáticamente.
3. Revisar la pestaña **Tests** del run y el resumen (porcentaje de aprobación).
4. Con el run exitoso, completar el Pull Request; el pipeline se vuelve a ejecutar sobre `main`.
5. Descargar el artefacto `servix-apk-debug` para las pruebas en dispositivo.

13.3 Pruebas pendientes

Tabla 14. Procedimiento de las pruebas pendientes

Prueba	Comando o procedimiento	Requisito previo
Android Lint (CP-044)	`gradlew.bat lintDebug`	Memoria suficiente.
Instrumentada (CP-045)	`gradlew.bat connectedDebugAndroidTest`	Dispositivo conectado con depuración USB.
Instalación (CP-047)	`adb install -r app-debug.apk`	APK descargado y dispositivo conectado.
Funcionales (CP-030 a CP-041, CP-046, CP-048)	Ejecutar los pasos de `Casos_Prueba.xlsx` con el APK instalado.	Cuentas de prueba de cada rol.
Reglas (CP-042)	Emulador de Firebase con pruebas de reglas, o ensayo en la consola.	Firebase CLI.
Make (CP-043)	Botón de prueba de conexión del Perfil.	URL de pruebas en `local.properties`.

Anexo A. Catálogo de casos de prueba

Tabla 15. Catálogo de casos de prueba (CP-001 a CP-066)

ID	Tipo	Caso	Requisito	Estado
CP-001	Unitaria	addition_isCorrect (`ExampleUnitTest`)	RNF-009	Aprobado
CP-002	Robolectric	read string from context (`ExampleRobolectricTest`)	RNF-009	Aprobado
CP-003	Robolectric	main activity launches without crash (`ExampleRobolectricTest`)	RF-005	Aprobado
CP-004	Robolectric	greeting_screenshot (`GreetingScreenshotTest`)	RF-005	Aprobado
CP-005	Unitaria	testInitialDataLoaded (`ServixUnitTests`)	RF-006, RF-010	Aprobado
CP-006	Unitaria	testClienteInitials (`ServixUnitTests`)	RF-006	Aprobado
CP-007	Unitaria	testAddCliente (`ServixUnitTests`)	RF-006	Aprobado
CP-008	Unitaria	testAddAndChangeStatusServicio (`ServixUnitTests`)	RF-010, RF-011	Aprobado
CP-009	Unitaria	testDeleteClienteCascade (`ServixUnitTests`)	RF-007	Aprobado
CP-010	Unitaria	testInitialDataLoaded (`ServixRepositoryTest`)	RF-006, RF-010	Aprobado
CP-011	Unitaria	testAddAndGetCliente (`ServixRepositoryTest`)	RF-006	Aprobado
CP-012	Unitaria	testAddAndUpdateServicioStatus (`ServixRepositoryTest`)	RF-010, RF-011	Aprobado
CP-013	Unitaria	testInitialUserAuthenticatedInMock (`ServixAuthUnitTests`)	RF-002	Aprobado
CP-014	Unitaria	testSignInSuccess (`ServixAuthUnitTests`)	RF-002	Aprobado
CP-015	Unitaria	testSignInValidationFailsOnShortPassword (`ServixAuthUnitTests`)	RF-002	Aprobado
CP-016	Unitaria	testSignUpSuccess (`ServixAuthUnitTests`)	RF-001	Aprobado
CP-017	Unitaria	testSignOut (`ServixAuthUnitTests`)	RF-003	Aprobado
CP-018	Unitaria	testCrearSolicitudAtencion (`ServixAtencionUnitTests`)	RF-019, RF-020	Aprobado
CP-019	Unitaria	testTomarSolicitudPorAdmin (`ServixAtencionUnitTests`)	RF-020	Aprobado
CP-020	Unitaria	testResponderSolicitudPorAdmin (`ServixAtencionUnitTests`)	RF-020	Aprobado
CP-021	Unitaria	testResolverSolicitud (`ServixAtencionUnitTests`)	RF-020	Aprobado
CP-022	Integración (doble)	testAddServicioDispatchesMakeWebhook (`ServixMakeIntegrationTest`)	RF-022	Aprobado
CP-023	Integración (doble)	testUpdateServicioEstadoCompletadoDispatchesMakeWebhook (`ServixMakeIntegrationTest`)	RF-011, RF-022	Aprobado
CP-024	Integración (doble)	testUpdateServicioCompletadoDispatchesMakeWebhook (`ServixMakeIntegrationTest`)	RF-022	Aprobado
CP-025	Robolectric	testAppLaunchAndDisplayHome (`ServixRobolectricTest`)	RF-005, RF-015	Aprobado

ID	Tipo	Caso	Requisito	Estado
CP-026	Robolectric	testNavigationToClientes (`ServixRobolectricTest`)	RF-005, RF-006	Aprobado
CP-027	Robolectric	testNavigationToServicios (`ServixRobolectricTest`)	RF-005, RF-010	Aprobado
CP-028	Robolectric	testNavigationToHistorial (`ServixRobolectricTest`)	RF-005, RF-012	Aprobado
CP-029	Robolectric	testNavigationToPerfil (`ServixRobolectricTest`)	RF-005, RF-016	Aprobado
CP-030	Funcional (dispositivo)	Inicio de sesión como Cliente	RF-002, RF-005	No ejecutado
CP-031	Funcional (dispositivo)	Inicio de sesión como Administrador	RF-002, RF-005	No ejecutado
CP-032	Funcional (dispositivo)	Rechazo por rol que no coincide	RF-002	No ejecutado
CP-033	Integración (Firebase)	Registro real y creación de usuarios/{uid}	RF-001	No ejecutado
CP-034	Integración (Firebase)	Recuperación de contraseña	RF-004	No ejecutado
CP-035	Funcional (dispositivo)	Gestión de clientes sobre Firestore	RF-006, RF-007, RF-008, RF-009	No ejecutado
CP-036	Funcional (dispositivo)	Gestión de servicios y estados sobre Firestore	RF-008, RF-010, RF-011	No ejecutado
CP-037	Funcional (dispositivo)	Notificaciones, historial y búsqueda global	RF-012, RF-013, RF-014	No ejecutado
CP-038	Integración (Gemini)	Respuesta del chat con Gemini	RF-018	No ejecutado
CP-039	Integración (Gemini)	Escalamiento por queja desde el chat	RF-019	No ejecutado
CP-040	Seguridad (dispositivo)	El cliente solo ve sus solicitudes	RF-021, RNF-001	No ejecutado
CP-041	Funcional (dispositivo)	Atención de solicitudes por el administrador en Firestore	RF-020, RF-021	No ejecutado
CP-042	Seguridad	Reglas de Firestore deniegan accesos no autorizados (`Manual / emulador de reglas`)	RNF-001	No ejecutado
CP-043	Integración (Make)	Webhook de Make real	RF-022	No ejecutado
CP-044	Calidad estática	Android Lint (`gradlew lintDebug`)	RNF-006	No ejecutado
CP-045	Instrumentada	Prueba instrumentada de ejemplo (`ExampleInstrumentedTest`)	RNF-009	No ejecutado
CP-046	Usabilidad (dispositivo)	Modo oscuro, contraste y legibilidad	RF-016, RNF-007	No ejecutado
CP-047	Instalación (dispositivo)	Instalación del APK debug (`adb install`)	RNF-005	No ejecutado
CP-048	Fiabilidad (dispositivo)	Falla de la IA sin conectividad	RF-018, RNF-008	No ejecutado
CP-049	Compilación	Compilación del APK debug (local) (`gradlew :app:assembleDebug`)	RNF-010	Aprobado
CP-050	CI	Ejecución del pipeline 20261101.3 (`Azure Pipelines ServiX-CI`)	RNF-009, RNF-010	Aprobado
CP-051	Render de UI (datos mock)	login (`ScreenshotsDocTest`)	RF-002, RNF-007	Aprobado
CP-052	Render de UI (datos mock)	registro (`ScreenshotsDocTest`)	RF-001, RNF-007	Aprobado
CP-053	Render de UI (datos mock)	home (`ScreenshotsDocTest`)	RF-005, RF-015, RNF-007	Aprobado

ID	Tipo	Caso	Requisito	Estado
CP-054	Render de UI (datos mock)	clientes (`ScreenshotsDocTest`)	RF-006, RNF-007	Aprobado
CP-055	Render de UI (datos mock)	clienteDetalle (`ScreenshotsDocTest`)	RF-006, RF-009, RNF-007	Aprobado
CP-056	Render de UI (datos mock)	servicios (`ScreenshotsDocTest`)	RF-010, RNF-007	Aprobado
CP-057	Render de UI (datos mock)	servicioDetalle (`ScreenshotsDocTest`)	RF-011, RNF-007	Aprobado
CP-058	Render de UI (datos mock)	historial (`ScreenshotsDocTest`)	RF-012, RNF-007	Aprobado
CP-059	Render de UI (datos mock)	perfil (`ScreenshotsDocTest`)	RF-016, RNF-007	Aprobado
CP-060	Render de UI (datos mock)	chatAdmin (`ScreenshotsDocTest`)	RF-018, RNF-007	Aprobado
CP-061	Render de UI (datos mock)	chatCliente (`ScreenshotsDocTest`)	RF-018, RF-019, RNF-007	Aprobado
CP-062	Render de UI (datos mock)	notificaciones (`ScreenshotsDocTest`)	RF-013, RNF-007	Aprobado
CP-063	Render de UI (datos mock)	nuevoCliente (`ScreenshotsDocTest`)	RF-006, RNF-007	Aprobado
CP-064	Render de UI (datos mock)	nuevoServicio (`ScreenshotsDocTest`)	RF-010, RNF-007	Aprobado
CP-065	Render de UI (datos mock)	portalCliente (`ScreenshotsDocTest`)	RF-021, RNF-007	Aprobado
CP-066	Render de UI (datos mock)	escalamientosAdmin (`ScreenshotsDocTest`)	RF-020, RNF-007	Aprobado